

```
# 9 - Clear the screen
window.fill(WHITE)

# 10 - Draw all screen elements
headerMessage.draw()
startButton.draw()
clickMeButton.draw()
timerMessage.draw()

# 11 - Update the screen
pygame.display.update()

# 12 - Slow things down a bit
clock.tick(FRAMES_PER_SECOND) # make pygame wait
```

Listing 13-3: A main program that uses an instance of the `Timer` class

Again, I've cut the setup code. Before the main loop starts, we create a `Timer` object ❶. When the user clicks Start, we call `oTimer.start()` ❷ to start the timer running.

Each time through the loop, we call the `update()` method of the `Timer` object ❸. There are two ways to know when the timer ends. The simple way is to check for this call returning `True`. The sample code in [Listing 13-3](#) uses this approach. Alternatively, if we specified a value for `callBack` in the `__init__()` call, when the timer finished, whatever was specified as the `callBack` value would be called back. In most cases, I would suggest using the first approach.

There are two advantages to using a `Timer` class. First, it hides the details of the timing code; you only create a `Timer` object when you want to, and you call the methods of that object. Second, you can create as many `Timer` objects as you wish, and each will run independently.

Displaying Time

Many programs will need to count and display time to the user. For example, in a game, the elapsed time might be constantly displayed and updated, or the user might have a set amount of time to complete a task,